

atnmCalc: 41052 Advanced Algorithms

Assignment 1

Adnan Islam

Introduction

atnmCalc is a calculator that takes an expression written in a LaTeX format, parses it with an extended version of the Shunting Yard algorithm and evaluates it. The program supports simple operators such as addition, subtraction, multiplication, division and exponentiation, but it also supports LaTeX commands such as (and unfortunately limited to) `\frac` and `\sqrt`.

What I built

The application uses C++20, CMake, ImGui, GLFW for windowing, nlohmann's json and tinyfiledialogs. The algorithm is present within the repository's src folder in the parser.cpp file at the `Parser::parseExpression` function.

I built the backend of the calculator, which includes:

- src/lexer.cpp
- src/lexer.h
- src/parser.cpp
- src/parser.h
- src/evaluator.cpp
- src/evaluator.h

Everything else is generated by AI.

The algorithm

This program uses an extended version of the Shunting Yard algorithm which traditionally takes an expression written in infix notation and converts it to postfix notation.

Infix notation is the notation we commonly use throughout math today. An example looks like $1 + 3 \times 4$ where the operator operates on the operands/atoms that are adjacent to it. Postfix notation, or otherwise known as reverse polish notation, is similar to infix notation except that operators come after the operands, and operators operate on the operands previous to the operator when reading from left to right. The equivalent postfix notation to the example I gave previously would look like $1\ 3\ 4\ \times\ +$.

The Shunting Yard algorithm can also produce an abstract syntax tree (AST), which is a data structure where every node is either an operand with an operator parent or is an operator with

children that it will use to turn into an operand itself when evaluating. Leaf nodes are numbers/variables that feed into its parent operator.

ASTs have uses all over computer science, but are mostly used in compilers. My implementation of the algorithm produces an AST made from the original expression, which is then evaluated to give a final answer.

How I've extended the algorithm

The algorithm was only designed for use with typical infix notation and not with atoms like $\frac{5}{6}$. The extension I've made is to allow for these new atoms to be parsed and evaluated. The extensions include allowing for unary minus and the `\sqrt` and `\frac` commands from LaTeX.

To support unary minus, the algorithm had to be extended to be able to differentiate between a binary minus and a unary minus. This was done by separating the algorithm into two branches, one for when the algorithm expects an operand and one for when the algorithm expects an operator. If the algorithm encounters a minus while expecting an operand, then it must be a unary minus.

General application design

Backend

The backend takes the expression, tokenises it with a lexer, applies the algorithm on the tokens in the parser and then evaluates the resulting AST with an evaluator.

The Lexer

The program needs something in between the actual expression that the user writes and the algorithm so that the algorithm can expect a standard form. This is what the lexer does. It takes the expression string and converts it into tokens. A simplified example of this working is with an expression like `"\sqrt{\frac{5*62}{3}}"` which then turns into an array of tokens which looks like `"SQRT, LBRACE, FRAC, LBRACE, 5, ASTERISK, 62, RBRACE, LBRACE, 3, RBRACE, RBRACE"`.

The lexer cleans the input of whitespace and rejects invalid characters and bad syntax and throws its own errors so that the parser can safely work on the tokens without worrying about any errors.

The Parser

This is where the actual algorithm is implemented in the application.

The parser class has 3 private fields: a private token list, a position variable that stores the parser's current position in the token list and a list of all the variable names.

The main parser function contains two stacks, one for the operand nodes and one for the operator nodes. It also contains a boolean flag called "expectOperand" which is used to separate the algorithm into the operand branch and the operator branch. These branches are where tokens are put onto either the operand or operator stacks.

With the operand branch, whenever the parser encounters an atom that has a subexpression such as a parenthesis or `\sqrt`, it calls the same parser function to create a sub-tree from the subexpression. This sub-tree then goes on the operand stack.

With the operator branch, before it puts an operator on the stack, it compares the precedence of the operator currently at the top of the stack and the 'incoming' operator and determines which operator should go before. An example of this would be for an expression like " $5 + 3 \times 4$ ". When $\times$ arrives, the parser compares it with the $+$ on top: $\times$ has higher precedence, so $+$ must wait. It isn't resolved, and $\times$ is pushed on top of it. Once the input ends, the stacks are drained from the top: $\times$ combines 3 and 4 into $\text{Mul}(3,4)$, then $+$ combines 5 and that node into $\text{Add}(5, \text{Mul}(3,4))$.

Before the parser can enter one of these branches, it checks that it is not at the end of an expression/sub-expression. If it is, then it will process the nodes in the stacks to produce the final AST that the function will return.

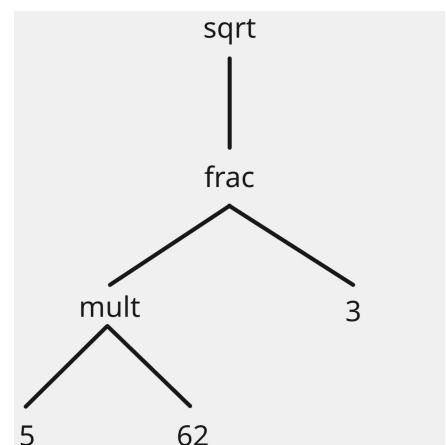
An example of what the parser will produce is if the input are the "SQRT, LBRACE, FRAC, LBRACE, 5, ASTERISK, 62, RBRACE, LBRACE, 3, RBRACE, RBRACE" tokens from the "`\sqrt{\frac{5*62}{3}}`" expression, then the algorithm will produce a tree that looks like "`Sqrt(Frac(Mul(5,62),3))`"

The Evaluator

The evaluator is where the AST from the parser gets turned into a single value to be presented by the frontend.

The evaluator is a recursive walk over the AST with a post order traversal. The number nodes and variable nodes are the base cases where they simply return their value. Variable nodes only store their name and they lookup their value with a variables map passed in from the frontend. With the rest of the node types, they evaluate their children, then apply the actual math operation and return it.

The evaluator is the only place where errors are caught. When they do get caught, the evaluator returns a failed evaluation with an error message that gets displayed by the frontend.



Frontend

Although the frontend was entirely created by the AI, almost all of the design decisions were made by me.

The following image is the initial design that I created for the frontend:

atnmCalc — □ ×

file settings

1

Enter equation: _____

(image/otherwise of rendered latex equation)

(drop down menu to define variables)

image or otherwise of rendered solution

And the following is the final design:

atnmCalc — □ ×

File Settings

Basic precedence

Enter equation: `2 + 3 * 4`

$$2 + 3 \times 4$$

(no variables in this equation)

Result

14

Sqrt of a fraction with multiplication

Enter equation: `\sqrt{\frac{d*2e}{3a}+5b}+c`

$$\sqrt{\frac{d \times 2 \times e}{3 \times a} + 5 \times b + c}$$

1.000

a

2.000

d

3.000

c

4.000

d

5.000

e

Result

7.83046

Left-associative subtraction

Enter equation: `10 - 2 - 3`

$$10 - 2 - 3$$

(no variables in this equation)

Result

5

Right-associative exponent

There are many design decisions here, but I will go through the most important.

1. Graphical interface.

This is mostly because I wanted the program to look nice and presentable, and also because I was inspired by Ilias' IliasDiff program. This kind of application could potentially be replicated in a console window and I am a very big fan of console applications, except you can't really render any images in the console, and I wanted to be able to render a preview of the expression

2. Multiple rows

This lets you have many equations at one time. Each row has its own state and variable list. Each row is renamable. The advantage of this decision is obvious; you don't need multiple instances of the calculator open to be able to calculate different expressions. This was inspired by Desmos.

3. Resizable UI panes

This lets you change the stretch and compress the panes to your liking. This is useful in the case of a very long fraction and you want to show the entire render on the screen at once. This decision was inspired by when I was trying to implement a UI scaling setting and setting the UI scaling really high made some panes an awkward size.

4. Instant expression evaluation

This lets you preview your work immediately instead of having a calculate button that you awkwardly have to click every time you want to evaluate your expression. It also lets you see errors immediately.

What I learned

Mistakes

- I tried implementing unary minus as $0 - x$ using a subtraction node in the parser. I had missed the UnaryMinusNode that was in the ast.h that the AI wrote. This was caught by the AI.
- In the lexer, I forgot to add a continue to the whitespace filter and so it pushed an unknown token for every whitespace. This was caught in the tests.
- In the lexer, I tried to check the length of a number by converting a double to a string and checking the length of the string. I did not know that converting a double to a string adds extra digits.

Surprises

- `std::stod("1.2.3")` doesn't throw but instead returns 1.2

- `std::stack::top()` on an empty stack doesn't throw but crashes the program with an access violation.
- Floating-point division by zero returning infinity is an interesting result to me. Same as `sqrt` of -1 returning NaN

New understandings

- I have a deeper understanding of writing algorithms with trees since I had been a bit turned away from trying to understand how they work since it seemed a little bit contradictory to me that a data structure defied the array-like structure that I was used to seeing.
- The shunting yard algorithm itself. This has given me a new insight into how computers parse things, which is something I've had a particular interest in ever since I wanted to make a calculator that didn't just take in one operand and operator at a time.
- Prefix/Infix/Postfix notation. This gives a new insight into writing expressions and it seems interesting how we as a society have decided to use the one method that introduces the most ambiguity into the order of operations when postfix and prefix notation have no such concept.
- `shared_ptr`. Memory management in general is something that fascinates me quite a bit (aha bits). I once heard about shared pointers and have finally been able to get a better understanding of how it works. I like how it gives me the comfort of automatic memory management yet the control of directly managing memory.

AI use

How AI was used

I have heavily used Claude both to guide me through writing the algorithm and for writing code not related to the backend such as the entire frontend, the tests, the cmakeLists.txt and the icon implementation.

For guiding me through writing the algorithm, it significantly contributed to designing the lexer and parser and gave me pseudo code for both of those. I also used AI to review code after I wrote it, and it gave me pointers on how to fix/finish the code.

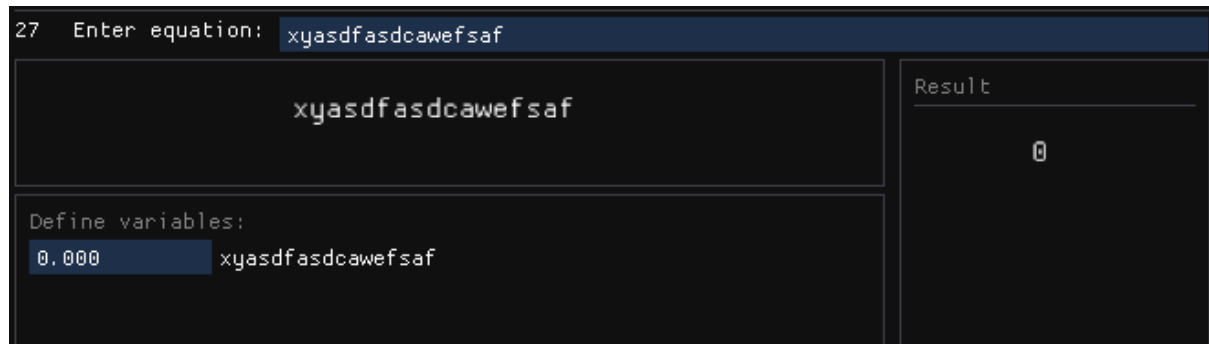
Where the AI was wrong

The first mistakes that Claude made were in the very first prototype where the AI made 2 mistakes. Those mistakes were that the square root symbol in the equation preview renderer was instead replaced by a question mark, as shown below.

16 Enter equation: <code>\sqrt{9}</code>	
$\sqrt{9}$ (no variables in this equation)	Result 3
17 Enter equation: <code>\sqrt{2}</code>	
$\sqrt{2}$ (no variables in this equation)	Result 1.41421
18 Enter equation: <code>\sqrt{-1}</code>	
$\sqrt{-1}$ (no variables in this equation)	Result <code>\sqrt</code> of a negative number

This was fixed with the radical being drawn as line segments instead of trying to use a character.

The next mistake that it made was that any string of continuous alphabetical characters registered as a single pronumeral instead of as separate pronumerals. This is shown below.



This was fixed immediately by the AI after I pointed it out. This was part of the AI's mock backend.

Another area where the AI made a mistake was in the design of the parser. In the parser, when the code goes to create a node for the abstract syntax tree, it pops the operands stack twice, once for the left operand, and once for the right operand. However, this caused a windows access violation in the case of an expression like "5 + " where there was no right operand on the stack. The AI itself says "applyTopOperator's code as I originally sketched it never checked operand count before popping — that omission is mine".

Parts of the AI output that I did not understand

The parts of the AI output that I do understand are the actual backend algorithm driving the calculator. I specifically told the AI not to give me the actual code and to guide me through writing and understanding it myself. As a result, I have a good understanding of the lexer, parser and evaluator, and especially with the evaluator, I wrote it myself with very little input from the AI.

Regarding what parts of the program I don't understand, it would be only the front end of the program which includes how to use ImGui, how the code walks through the AST that comes from the parser to make a visual representation of the equation. The tests, the CMakeLists file, the icon implementation and the readme are also fully written by the AI and I don't and have not tried to understand how these are implemented, although if I want to continue working on this project, I should probably understand how those systems work.

References

Shunting yard algorithm. (2026, 28 September)

https://en.wikipedia.org/w/index.php?title=Shunting_yard_algorithm&oldid=1361964542

Anthropic. (2026, 28 September). LaTeX math equation calculator [Generative AI chat]. Claude Sonnet 5, Sonnet 5.5, Opus 5.5. claude.ai